Kodutöö 6 - Andmete hoidmine ja ligipääs mobiiliseadmes

Töö eesmärk

Selle kodutöö eesmärk oli salvestada rakenduse andmed telefoni mällu nii, et need oleksid alles ka pärast rakenduse sulgemist. Salvestame kolme asja: kasutaja nime ja e-posti, rakenduse teema valiku ning kaameraga tehtud foto.

Jätkasime React Native'i ja Expo abil loodud grupirakenduse arendamist. **Seadete**, **Profiili** ja **Meedia** leht olid rakenduses juba varasematest kodutöödest olemas: seadetes sai valida teemat ja Meedia lehel sai kaameraga pildistada. Telefonis ei jäänud aga midagi meelde. Pärast rakenduse sulgemist olid teema valik ja foto kadunud.

Mida tegime?

1. Kasutaja profiil seadete lehel

Seadete lehele lisasime uue jaotise **Kasutaja profiil** kahe tekstiväljaga: **Nimi** ja **E-post**. Nupp **Salvesta muudatused** salvestab need seadme mällu ja näitab teadet „Kasutaja andmed salvestati seadme mällu!“ Lehe uuesti avamisel on väljad juba täidetud.

Profiili leht näitas varem alati sama teksti „Kasutaja Profiil“ ja „Grupp 5 testkasutaja“. Nüüd kuvab see salvestatud nime ja e-posti. Kui midagi pole salvestatud, jääb vana tekst.

2. Kaamerafoto salvestamine telefoni mällu

Varem kuvati Meedia lehel tehtud foto ainult seni, kuni rakendus oli lahti. Nüüd kopeeritakse iga tehtud foto rakenduse püsivasse kausta failina **salvestatud_foto.jpg** ja kasutaja näeb teadet „Foto salvestati seadme lokaalsesse mällu.“ Kui rakendus uuesti avada, kuvatakse viimati salvestatud foto kohe Meedia lehel.

3. Teema valik jääb telefonis meelde

Teema valik (hele, tume või süsteemi järgi) oli seadetes juba olemas, kuid see jäi meelde ainult veebiversioonis. Muutsime teemahaldust nii, et valik salvestatakse ka telefonis.

Mida ja kuhu salvestame?

Andmed	Kuidas salvestame	Kus koodis
Nimi ja e-post	AsyncStorage, võtmed `@user_profile_name` ja `@user_profile_email`	src/app/settings/index.tsx, src/app/profile/index.tsx
Teema valik	AsyncStorage, võti `grupp5_theme_preference`	src/context/theme-context.tsx
Kaamera foto	Fail `salvestatud_foto.jpg` rakenduse dokumentide kaustas (expo-file-system)	src/app/media/index.tsx

Selleks lisasime projekti kaks teeki: **@react-native-async-storage/async-storage** ja **expo-file-system**.

Kuidas kood töötab?

Nimi, e-post ja teema AsyncStorage'is

AsyncStorage hoiab lihtsaid tekstiväärtusi võtme ja väärtuse paaridena, sarnaselt brauseri localStorage'iga, aga töötab ka telefonis. Kõik tegevused on asünkroonsed, seega kasutame `await` märksõna.

Seadete lehe avamisel loeb `useEffect` funktsiooniga `AsyncStorage.getItem()` varem salvestatud nime ja e-posti ning paneb need tekstiväljadesse. Nupule vajutades kirjutab `AsyncStorage.setItem()` väljade sisu mällu. Profiili leht kasutab `useFocusEffect`-i, mis loeb andmed uuesti iga kord, kui kasutaja lehele tagasi tuleb. Nii on muudatus profiilis kohe näha.

Teema valik salvestatakse kohe, kui kasutaja selle vahetab. Rakenduse käivitamisel loeb teemahaldus salvestatud valiku ja rakendab selle kogu rakendusele.

Foto ja expo-file-system

Foto salvestamiseks kasutame expo-file-system'i klasse `File` ja `Paths`. `Paths.document` on rakenduse dokumentide kaust, mis jääb alles ka pärast rakenduse sulgemist ja telefoni taaskäivitamist.

Kaamera annab pildi ajutise faili aadressi. Kood loob sellest objekti `new File(asset.uri)`, kustutab vajadusel eelmise salvestatud foto ja kopeerib uue meetodiga `copy()` faili `salvestatud_foto.jpg`. Meedia lehe avamisel kontrollib kood omadusega `exists`, kas fail on olemas, ja kui on, kuvab selle. Pildi aadressi lõppu lisame `?time=` koos ajatempliga, et telefon ei näitaks sama nimega faili vana versiooni vahemälust.

Mis osutus lihtsaks?

AsyncStorage'i ja expo-file-system'i kasutamine oli lihtne: väärtuse salvestamiseks ja lugemiseks piisab ühest käsust ning faili kopeerimiseks samuti. Mõlemad teegid töötavad Expo Go's, seega development buildi polnud vaja. Kuna lehed olid juba olemas, tuli neile lisada ainult salvestamise ja lugemise loogika.

Mis osutus keerulisemaks?

- **Veebiversioon jooksis kokku veaga** `this.validatePath is not a function`. Foto faili objekt loodi faili ülaosas, väljaspool komponenti. Expo Router renderdab veebi jaoks lehed esmalt serveris, kus telefoni failisüsteemi pole. Tõstsime faili loomise eraldi funktsiooni `getSavedPhoto()`, mida kutsutakse alles siis, kui faili päriselt vaja on.
- **Vana pilt vahemälus.** Kuna fail kannab alati sama nime, võis telefon pärast uue foto tegemist näidata vana pilti. Selle lahendas aadressile lisatud `?time=` parameeter.